TRIALX CONNECT — CLINICAL TRIALS

Front-End Test Strategy

Trial Search & Study-Contact “Send a message” Form

Author	Auqid Irfan
Application	TrialX Connect — Clinical Trials listings
Test environment	https://dev-connect.ainfo.io/clinical-trials/listings/
Date	04 Jul 2026
Status	Everything here was checked against the live dev site with Playwright — selectors, field limits, and validation messages reflect what the app actually does today.

1. Objective

This document sets out how I'd test two parts of the TrialX clinical-trials site: the trial search (landing search, results, filtering, sorting, pagination, and the empty state) and the study-contact “Send a message” flow (site selection → contact form → validation → confirmation). The goal is to confirm both work correctly, hold up against bad input, and behave the same across the browsers we support.

I wrote it to be picked up and used straight away — for manual runs and for Playwright automation. Rather than guess at the markup, I drove the live dev site and read the DOM directly, so every selector, field limit, and validation message below is what the app actually does today. Where I couldn't confirm something, I've flagged it rather than state it as fact (see “Open items to confirm” above).

2. Scope

What I'm testing, and what I'm deliberately leaving out so the suite stays focused on the front end.

In scope

- Landing search: condition/keyword field (#search-medical-conditions), location field (#search-study-location), condition autocomplete, form submission and URL contract.
- Results page: result cards, sort control, location + radius filter, advanced filters, facets, pagination, and the no-results empty state.
- Study-contact flow on a featured trial (/clinical-trials/listings/<id>/): the multi-step “Find a Site” widget → site selection → “Send a message” form → client-side validation → success confirmation.
- Client-side field constraints (required, maxlength, email format, phone), consent enforcement, and post-submission state.
- Cross-browser rendering/behaviour and basic responsive checks; accessibility smoke checks (labels, keyboard, focus).
- Front-end security smoke: reflected/echoed-input rendering and DOM safety (not a formal security audit).
- Front-end performance smoke: page/search timing and loader behaviour, incl. throttled network (not load/stress testing).

Out of scope

- The /clinicaltrial/<id>/ microsite template (marketing “study website” with prescreener) — different template, no standard contact form. Automation must not land on these.
- Back-end / API correctness of search ranking, geo-resolution, and message delivery (covered by API/integration suites).
- Google Maps / Places third-party internals (only our integration points are tested); OneTrust cookie-consent internals (we only assert accept/dismiss).
- Heavy load/stress testing, penetration testing, and formal security audits (separate specialised efforts).
- Email/SMS deliverability and CRM ingestion.

3. Test Approach & Levels

How I'd layer the testing, the techniques behind the case design, and the concrete facts I pulled off the live site that the automation depends on.

Levels

Level	Focus	Owner
Component / UI unit	Individual widgets (autocomplete, radius select, consent checkboxes) behave per spec	Dev + QA
Integration (front-end)	Search form → results URL contract; site selection → contact form → success screen	QA automation
System / E2E	Full journey: landing → search asthma → open featured trial → select site → send message → confirmation	QA automation
Cross-browser / responsive	Rendering + interaction across the browser matrix and breakpoints	QA
Accessibility smoke	Labels, keyboard operability, focus order, error announcement	QA

Techniques

- Equivalence partitioning & boundary-value analysis for input fields (maxlength 30/150 boundaries, autocomplete threshold = 3).
- Data-driven testing to fold input-only variations (valid/invalid email, required-field permutations, search-term families) into single parameterised cases.
- State-transition testing for the multi-step site widget (select → next → validate → submit → success / already-contacted).
- Negative & error-guessing for validation, reflected input, and template-routing edge cases.

Key DOM-verified facts the automation relies on

These came off the live DOM, and the automation keys on them. A couple differ from what the markup alone would suggest — worth pinning here so nobody re-guesses them.

- Landing search form: action="/clinical-trials/listings/search/", GET; submits q, place, and hidden geo_lat, geo_lng, user_country. Both visible fields are optional (no HTML5 required); autocomplete="off".
- Condition autocomplete: .bootstrap-autocomplete.dropdown-menu → a.dropdown-item; minimum 3 characters (hidden at 1–2). #search_medical_list is the browse-by-condition list, not the autocomplete.

- Results sort select[name="sort_by"]: relevance, distance, last_updated, most_viewed. (Confirm exact keys.)
- Radius #location-radius-dropdown (name="radius"): 10, 25, 50, 100, 250, 11000 (Entire World). (Confirm 11000.)
- Results paging: 10 .result-item cards per page; .pagination .page-link controls render as JS-driven elements.
- No-results state: renders No clinical trials found for '<query>'. with the query echoed back.
- Contact form lives in a multi-step “Find a Site” widget (#site-widget / #id_site_locator_widget): Select a site → Next → Send a message (#contact-site-form) → submit → #contact-site-success (.fas-success-screen, injected post-submit); the site tab switches to “Submitted” (#success-message-tab).
- Validation is client-side JS using Bootstrap is-invalid + .invalid-feedback. Verified messages: empty → “Required”; malformed email → “Invalid email address”. Invalid submit is blocked (stays on the form; no success screen).
- Cookie consent: OneTrust; accept via #onetrust-accept-btn-handler. “Already-contacted” state persists per site/trial (#already-contacted-site).
- Site-slot caps surface as widget messages: “maximum of 2 slots per day” and “up to 5 slots” when the limit is hit.

Contact fields (all required, JS-validated — no HTML5 required):

Field	Selector	Type	maxlength
First name	input[name="firstname"]	text	30
Last name	input[name="lastname"]	text	30
Email	input[name="email"]	email	150
Phone	#phone-no (name="phone-no")	tel (intl-tel-input, +1 default)	—
Age consent	#age_consent_checkbox	checkbox (required)	—
Terms consent	#cw-terms-conditions	checkbox (required)	—

4. Automation Approach

Playwright with a Page Object Model. A few of these choices come straight from working with the real widget — the MUI classes are unstable, the cookie banner blocks clicks, and “already-contacted” state can hide the form — so I've built around them rather than discovering them mid-run.

- Tool: Playwright, TypeScript, Page Object Model.
- Structure: page objects for LandingSearch, SearchResults, TrialDetail, SiteWidget, ContactForm; fixtures for cookie-consent handling and featured-trial navigation.
- Cookie consent: a global setup/fixture accepts OneTrust once (#onetrust-accept-btn-handler) and persists storage state, so the banner never intercepts later clicks; all specs reuse the saved state.
- Selector policy: prefer the stable IDs/names verified here; avoid MUI hashed classes (jss*) — drive the site widget by role/name (e.g. getByRole('button', { name: 'Next' }), site cards by accessible name).
- Waiting: web-first assertions / waitFor on .result-item and on #trialsLoader disappearing; never fixed sleeps.

- Data-driven: parameterised specs for the email-format and required-field matrices and the search-term families.
- Featured-trial guard: helper asserts the detail URL matches /clinical-trials/listings/<id>/ and that #contact-site-form exists; fail fast if redirected to a /clinicaltrial/<id>/ microsite.
- Isolation: because “already-contacted” state persists, each contact-submission E2E uses a fresh trial/site or a dedicated test trial.
- Reporting: HTML report + traces/screenshots/video on failure; run in CI on the browser matrix.

5. Test Data Strategy

Data set	Values	Purpose
Condition search terms	asthma (hits); as/ast (autocomplete 2 vs 3); zzxqwventyfake123 (no results); multi-word & special: type 2 diabetes, Ewing's, “Rhinitis, Allergic, Perennial”, MPS II	Happy path, autocomplete threshold, empty state, encoding
Reflected/injection probes	x, "><img src=x>, asthma' OR '1'='1	Query-echo escaping; literal handling
Names	Valid: Jane, 30-char string; boundary 30 vs 31 (expect cap at 30)	maxlength boundary
Email	Valid: qa+test@example.com; invalid: notanemail, a@, a@b, “a b@c.com”; 150-char boundary	Email format + maxlength
Phone	Valid US 2015550123; invalid 123; non-numeric	Phone / intl-tel validation
Consent	Both unchecked / one checked / both checked	Consent enforcement
Location & radius	Real city (New York) via Places; empty; tampered radius=999999 / abc	Radius + distance sort; URL tampering

- Use synthetic, non-PII data only (example.com, reserved +1 555… ranges).
- Prefer known test trials on the featured template for site-widget flows; centralise trial IDs in fixtures.
- Never submit real contact details to live sites; treat the dev message pipeline as observable only through the UI success screen.

6. Environment & Browser Matrix

Environment: DEV (dev-connect.ainfo.io). Pre-req: cookie consent accepted & stored; Google Maps/Places reachable.

Browser	Versions	Priority	Notes
Chromium (Chrome/Edge)	Latest + latest-1	P1	Primary automation target
Firefox	Latest	P2	Cross-engine
WebKit (Safari)	Latest	P2	Safari parity; intl-tel + Maps quirks
Mobile Chrome (Pixel)	Emulated	P2	Responsive, map/list toggle, touch
Mobile Safari (iPhone)	Emulated	P3	iOS input/keyboard behaviour

Breakpoints: mobile (≤576px), tablet (768px), desktop (≥1200px). Networks: normal broadband and throttled (Slow 3G) for the performance-smoke cases.

7. Risks & Mitigations

The things most likely to make these tests flaky or point at the wrong thing, and how I'd handle each. Most of these I hit while exploring the live site.

#	Risk	Impact	Mitigation
R1	Two detail templates; /clinicaltrial/<id>/ microsite has no contact form	Tests target the wrong page; false failures	URL guard + assert #contact-site-form before contact steps
R2	MUI hashed classes (jss*) are unstable	Flaky selectors	Drive widget by ARIA role/name; request stable data-testid (dev ask)
R3	"Already-contacted" state persists per site/trial	Contact form hidden → false failure	Fresh trial/site per run; detect #already-contacted-site
R4	Google Maps/Places latency or quota	Flaky site widget & location autocomplete	Web-first waits, retries, mock/stub Places where feasible
R5	Validation is JS-only (no HTML5 required)	Native-constraint assumptions fail	Assert on is-invalid / .invalid-feedback text, not :invalid
R6	Cookie banner intercepts clicks	Click failures mid-flow	Accept once in global setup; persist storage state
R7	Slot caps (2/day, 5 total) enforced app-side	Hard to reproduce deterministically	Dedicated data; treat as targeted, low-frequency case
R8	Query reflected in the empty-state message	Potential XSS if unescaped	Reflected-input smoke case (TC-S07 / TC-SEC02)
R9	Dev data is volatile (test trials renamed/removed)	Broken fixtures	Pin resilient search terms; centralise trial IDs in fixtures

8. Entry & Exit Criteria

Entry

- DEV build deployed and reachable; search index populated (asthma returns ≥1 page).
- At least one featured trial with ≥1 selectable site available.
- Cookie-consent storage state generated; Playwright suite green on smoke.
- Test data (Section 5) and this strategy reviewed.

Exit

- 100% of P1 cases executed; 0 open P1 defects.
- ≥95% of P2 cases executed; no open Critical/High defects.
- All required-field and email-format data-driven variants pass.
- Full happy-path E2E (search → featured trial → site select → send message → #contact-site-success) passes on all P1 browsers.
- Known issues documented with severity and workaround; results signed off.

9. Test Cases

Priority: P1 critical · P2 high · P3 medium. Type: F Functional · V Validation · UI/Responsive · A11y · N Negative · Sec Security-smoke · Perf Performance. ⚠ marks an item to confirm against the live DOM before sign-off.

9.1 Trial Search

ID	Title	Type	Pri	Preconditions	Test Data	Steps	Expected Result
TC-S01	Landing → results happy path & URL contract	F	**P1**	Cookie accepted; on landing page	q=asthma	1. Type asthma in #search-medical-conditions 2. Submit "Find trials"	– Navigates to /clinical-trials/listings/search/?q=asthma&place=&geo_lat=&geo_lng=&user_country= – Results grid renders; title reflects asthma; ≥1 .result-item shown
TC-S02	Condition autocomplete min-length threshold	F	**P1**	On landing page	a, as, ast	1. Type 1 char, wait 2. Type 2nd char, wait 3. Type 3rd char, wait	– Dropdown hidden at 1 & 2 chars – At exactly 3 chars, .bootstrap-autocomplete.dropdown-menu.show appears with a.dropdown-item suggestions
TC-S03	Selecting an autocomplete suggestion drives the search	F	**P2**	On landing page	asth → "Asthma"	1. Type asth 2. Click the "Asthma" suggestion 3. Submit	Field populated from the suggestion; results page shows asthma trials
TC-S04	Empty search submission	N	**P2**	On landing page	none	1. Leave both fields blank 2. Submit	– Form submits (fields are optional); app returns default/all-listings or a graceful prompt – No JS error, no blank crash ⚠ *CONFIRM: which behaviour the app actually shows (all-listings vs prompt)*
TC-S05	Sort control changes result ordering (data-driven)	F	**P1**	On results page for asthma	sort_by ∈ { relevance, distance, last_updated, most_viewed }	1. For each option, select it in select[name="sort_by"] 2. Await reload	– Results reload without error for each option – distance requires a location and orders by proximity when one is set ⚠ *CONFIRM: exact key strings last_updated / most_viewed*
TC-S06	Location + radius filter	F	**P2**	On results page	place=New York; radius ∈ { 10, 50, Entire World }	1. Enter location via Places autocomplete 2. Choose radius in #location-radius-dropdown 3. Apply	– radius reflected in URL/query – Result set updates consistently with the chosen radius

ID	Title	Type	Pri	Preconditions	Test Data	Steps	Expected Result
TC-S07	No-results empty state + reflected-input escaping	N / Sec	**P1**	On results page	q=zzxqwventyfake123; probe q=x and "><img src=x>	1. Search a gibberish term 2. Search the HTML probe	– Shows: No clinical trials found for '<query>'. with 0 cards – The echoed probe is rendered escaped — no tag executed or injected
TC-S08	Pagination navigation	F	**P2**	Results with >1 page (e.g. asthma)	pages 1→2, next/prev	1. Note 10 cards on page 1 2. Click page 2 / next 3. Click prev / page 1	– Each page shows up to 10 .result-item; active page indicated – #trialsLoader shows then clears; content changes per page – Note: .page-link renders as a JS-driven — assert on changed results, not a hard navigation
TC-S09	Advanced filters & facets apply / reset	F	**P2**	On results page	Gender / condition facet	1. Open #advanced-filter-modal 2. Select a facet 3. #apply-advanced-filters 4. #clear-all-facets / #reset-advanced-filters	– Facet narrows results – #clear-all-facets is hidden (d-none) until a facet is active, then clears back to the full set
TC-S10	Open a FEATURED result (template guard)	F	**P1**	On results page for asthma	featured trial (/clinical-trials/listings/<id>/)	1. Open a featured .search-item--link 2. Assert URL + form presence	– Lands on /clinical-trials/listings/<id>/; standard detail renders; #contact-site-form exists – Microsite /clinicaltrial/<id>/ must be skipped by the fixture
TC-S11	Results responsive rendering	UI	**P3**	Results page	mobile / tablet / desktop	1. Load results at each breakpoint	Cards, sort, and filter controls reflow without overlap/clipping; no horizontal scroll
TC-S12	Search accessibility smoke	A11y	**P3**	Landing + results	keyboard only	1. Tab to condition field 2. Operate autocomplete via keyboard 3. Submit	– Fields have labels; autocomplete options reachable/selectable by keyboard – Visible focus; results announced
TC-S13	Case-insensitive search (data-driven)	F	**P2**	On landing page	q ∈ { asthma, ASTHMA, AsThMa }	1. Search each casing variant in turn 2. Compare results and the retained field value	– All variants reach results and are not blocked by case – Result behaviour is consistent across casings
TC-S14	Whitespace is trimmed / normalised	N	**P2**	On landing page	q = “ asthma ” and “type 2 diabetes”	1. Enter the padded term and submit 2. Inspect the resulting URL and the field	– Query is trimmed / collapsed so the URL is well-formed – Results load without error

ID	Title	Type	Pri	Preconditions	Test Data	Steps	Expected Result
TC-S15	Multi-word & special-character terms encode correctly (data-driven)	F	**P2**	On landing page	q ∈ { “type 2 diabetes”, “Ewing's”, “Rhinitis, Allergic, Perennial”, “MPS II”, “non-small-cell lung cancer” }	1. Enter each term and submit 2. Inspect the q-parameter encoding and the results page	– Spaces, commas, and apostrophes are percent-encoded (no broken URL) – Results render with the term preserved; no JS error
TC-S16	Robustness: over-long / unicode / special-only queries (data-driven)	N	**P3**	On landing page	q ∈ { 200+ char string, emoji/unicode, “@@@###”, whitespace-only }	1. Enter each edge value and attempt to search 2. Observe validation, layout, and console	– Input is capped or returns a graceful no-results state – No crash, no layout break, no uncaught error
TC-S17	Injection-style string is treated as a literal search	Sec	**P2**	On landing page	q = asthma' OR '1'='1	1. Submit the string 2. Observe results and any error output	– The string is treated as a literal search term – No error message or stack trace is leaked to the UI
TC-S18	Tampered radius via direct URL is handled	Sec / N	**P3**	Results page reachable	radius=999999 ; radius=abc (via URL)	1. Load the results URL with each tampered radius 2. Observe results and console	– Value is clamped, ignored, or defaulted – No crash and no server error surfaced to the user

9.2 Study-Contact “Send a message” Form

ID	Title	Type	Pri	Preconditions	Test Data	Steps	Expected Result
TC-C01	Reach contact form via site selection	F	**P1**	– On a featured trial with ≥1 site; step 1 expanded – Cookie-consent (OneTrust) accepted	site = first available	1. In #site-widget, select a site 2. Click Next	– “Send a message” form (#contact-site-form) renders with First/Last/Email/Phone + two consent checkboxes – Selected site is shown
TC-C02	Successful submission (happy path)	F	**P1**	On contact form; fresh (not already-contacted) site	valid name/email/phone; both consents checked	1. Fill all fields validly 2. Check both consents 3. Submit	– #contact-site-success (.fas-success-screen) shows “Message sent successfully.” – Site tab switches to “Submitted” (#success-message-tab); no error – Note: #contact-site-success is injected only post-submit — never assert it on load
TC-C03	Required-field validation (data-driven)	V	**P1**	On contact form	Omit each of firstname/lastname/email/phone in turn; also all-empty	1. Submit with the target field(s) empty	– Submission blocked (form persists, no success) – Each empty required field flagged is-invalid with inline “Required” (JS validation; no HTML5 required)

ID	Title	Type	Pri	Preconditions	Test Data	Steps	Expected Result
TC-C04	Email format validation (data-driven)	V	P1	On contact form; other fields valid, consents checked	notanemail, a@, a@b, "a b@c.com"	1. Enter each invalid email 2. Submit	– Blocked each time; email shows inline "Invalid email address" – No success screen
TC-C05	Consent checkboxes are mandatory	V	P1	On contact form; all text fields valid	both-unchecked / age-only / terms-only	1. For each combination, submit	– Submission blocked unless both #age_consent_checkbox and #cw-terms-conditions are checked – Appropriate blocked/error state shown
TC-C06	Text-field maxlength boundaries	V	P2	On contact form	firstname/lastname 30 vs 31; email 150 vs 151	1. Enter boundary-length values (type and paste)	– Input caps at maxlength (30 / 30 / 150); 31st/151st char rejected; no overflow error ⚠ *CONFIRM: maxlength also blocks paste, not only typing*
TC-C07	Phone (intl-tel-input) validation	V	P2	On contact form	valid 2015550123; invalid 123, letters; alt country code	1. Enter values with default +1 and an alternate country	– Valid number accepted; malformed number blocked with a phone error – Country selector updates the dial code
TC-C08	Error clears on correction	V	P2	Contact form showing errors (from TC-C03/04)	valid replacements	1. Correct each flagged field 2. Re-submit	– is-invalid / messages clear on valid input – Submission proceeds to success
TC-C09	Site slot caps enforced	F	P3	Ability to select multiple sites	select >2 sites in a day; >5 total	1. Attempt to exceed the daily (2) and total (5) slot limits	– Widget prevents exceeding the caps and surfaces the message ("maximum of 2 slots per day" / "up to 5 slots") – Messages surface during site selection, not as static copy
TC-C10	Already-contacted site state	F	P3	A site previously contacted for this trial	previously submitted site	1. Reopen the trial / site	– #already-contacted-site state shown – The contact form is not re-presented for that site (no duplicate submission path)
TC-C11	Back navigation preserves widget state	F	P3	On contact form (after Next)	—	1. Click Back to site selection 2. Return via Next	– Returns to site list with prior selection intact – Re-entering the form does not unexpectedly lose already-entered valid data
TC-C12	Contact form responsive & keyboard a11y	UI / A11y	P3	Contact form	mobile / tablet / desktop; keyboard only	1. Render at each breakpoint 2. Tab through fields, toggle consents, submit via keyboard	– Layout reflows cleanly; all fields/labels associated – Consents toggle via keyboard; focus moves to error/success on submit

ID	Title	Type	Pri	Preconditions	Test Data	Steps	Expected Result
TC-C13	Cookie banner does not block flow	F	**P2**	First visit, banner present	—	1. Accept via #onetrust-accept-btn-handler 2. Complete a contact submission	– After accept, no overlay intercepts clicks on the step toggle or Next/Submit – Flow completes

9.3 Security & Performance (front-end smoke)

ID	Title	Type	Pri	Preconditions	Test Data	Steps	Expected Result
TC-SEC01	Injection / formula payloads in contact fields are neutralised (data-driven)	Sec	**P2**	On contact form	field values ∈ { <script>alert(1)</script>, "><img src=x onerror=alert(1)>, =1+1, +cmd, control chars }	1. Enter payloads into name/email/other fields 2. Submit where the form allows and observe rendering/echo	– No script executes; formula prefixes are neutralised; control chars handled – Values are escaped wherever echoed
TC-SEC02	Search reflected-input & injection handling (cross-ref)	Sec	**P2**	On landing / results page	see TC-S07, TC-S17, TC-S18	1. Execute the reflected-input, literal-injection, and URL-tamper checks	– Query echo is escaped; injection strings are literal; tampered params are clamped/ignored – No script execution, no error/stack-trace leakage
TC-PERF01	Results render within the agreed threshold; loader behaves	Perf	**P3**	Results reachable	Threshold per agreed target; normal + throttled (Slow 3G)	1. Search asthma and measure time to first .result-item on normal and throttled networks 2. Observe #trialsLoader appearing then clearing	– Results render within the agreed target on a normal network – Loader shows during fetch and clears when results arrive; throttled network degrades gracefully
TC-PERF02	Autocomplete / search under slow or failed network	Perf	**P3**	On landing / results page	throttle or block autocomplete and search requests	1. Throttle/block the requests and interact with the fields	– No hang or crash; graceful handling – Fields stay usable and the user can still submit

10. Deliverables & Traceability

- This strategy document (living; update as the dev app changes).
- Playwright suite mapped 1:1 to the TC IDs above (spec/test title references the ID).
- CI report with per-browser results, traces, and screenshots on failure.
- Defect log with severity, browser, and reproduction from the relevant TC.